

Chapter 10

Github Repository Link - <https://github.com/reim7/CS4080-HW5>

1. **Our interpreter carefully checks that the number of arguments passed to a function matches the number of parameters it expects. Since this check is done at runtime on every call, it has a performance cost. Smalltalk implementations don't have that problem. Why not?**

Smalltalk uses a different method-calling structure than Lox. Instead of separating arguments with commas, Smalltalk uses keyword selectors, where each argument has its own part of the method name.

For example, a method call might look like `list insert: "element" at: 2`. In this case, `insert:at:` is the complete method name, and its structure already specifies how many arguments it accepts.

Because the number of arguments is built into the method's name, passing a different number of arguments would produce a different selector. Smalltalk therefore doesn't need a separate runtime check to verify the argument count like Lox does.

2. **Lox's function declaration syntax performs two independent operations. It creates a function and also binds it to a name. This improves usability for the common case where you do want to associate a name with the function. But in functional-styled code, you often want to create a function to immediately pass it to some other function or return it. In that case, it doesn't need a name.**

Languages that encourage a functional style usually support anonymous functions or lambdas—an expression syntax that creates a function without binding it to a name. Add anonymous function syntax to Lox so that this works:

```
fun thrice(fn) {  
  
    for (var i = 1; i <= 3; i = i + 1) {  
  
        fn(i);  
  
    }  
  
}
```

```

thrice(fun (a) {

    print a;

});

// "1".

// "2".

// "3".

```

How do you handle the tricky case of an anonymous function expression occurring in an expression statement:

```
fun () {};
```

For this challenge, I modified the Lox interpreter to support anonymous functions. I updated the syntax tree, parser, and interpreter so functions can be created without names, passed as arguments, and executed. Named functions continue to work as before.

3. Is this program valid?

```

fun scope(a) {

    var a = "local";

}

```

In other words, are a function's parameters in the *same* scope as its local variables, or in an outer scope? What does Lox do? What about other languages you are familiar with? What do you think a language *should* do?

In Lox, function parameters and local variables declared directly inside the function body share the same scope. This means a programmer cannot declare a local variable with the same name as an existing parameter.

This would produce an error because the name `a` has already been declared in that scope.

Other programming languages handle this differently. Java and C generally prevent parameters from being redeclared in the same function scope, while Dart allows local variables to shadow parameters because they are treated as belonging to different scopes.

I prefer keeping parameters and local variables in the same scope because it makes code easier to understand. Allowing two variables with the same name inside a function could create confusion about which value is being used. Preventing this type of redeclaration helps programmers avoid unnecessary mistakes.

Chapter 11

1. Why is it safe to eagerly define the variable bound to a function's name when other variables must wait until after they are initialized before they can be used?

It is safe to define a function's name before resolving its body because the code inside a function does not execute immediately when the function is declared. It only executes when the function is called.

Defining the function's name first also allows recursion, where a function calls itself.

Variables are different because their initializer expressions are evaluated immediately. If a variable tries to reference itself before initialization is complete, it could access a value that has not been assigned yet.

Therefore, defining function names early is safe and necessary for recursion, while variables must be initialized before they can be accessed.

2. How do other languages you know handle local variables that refer to the same name in their initializer, like:

```
var a = "outer";
```

```
{
```

```
    var a = a;
```

```
}
```

Is it a runtime error? Compile error? Allowed? Do they treat global variables differently? Do you agree with their choices? Justify your answer.

Different programming languages have different rules for variables that reference themselves during initialization.

In C, a local variable becomes visible before its initializer is evaluated. This means it may reference itself before receiving a value, which can lead to undefined behavior.

Java is stricter. It does not allow a local variable to be redeclared within an overlapping local scope. If the outer variable is a class field instead, Java's definite assignment rules still prevent the new local variable from reading itself before initialization.

Lox also detects this problem through its resolver and reports an error when a local variable attempts to read itself during initialization.

I prefer Lox's approach because it identifies the mistake before the program runs. This prevents unexpected behavior and encourages programmers to write clearer code.

3. Extend the resolver to report an error if a local variable is never used.

For this challenge, I modified the resolver to identify local variables that are declared but never used. The resolver tracks whether each variable has been declared, defined, or read.

When a scope ends, the resolver checks the variables declared within it. If a variable was defined but its value was never read, the program reports an error.

This helps identify unnecessary variables and potential programming mistakes before the program executes.

4. Our resolver calculates *which* environment the variable is found in, but it's still looked up by name in that map. A more efficient environment representation would store local variables in an array and look them up by index.

Extend the resolver to associate a unique index for each local variable declared in a scope. When resolving a variable access, look up both the scope the variable is in and its index and store that. In the interpreter, use that to quickly access a variable by its index instead of using a map.

For this challenge, I modified the resolver and interpreter to store local variables using array indexes instead of looking them up by name. The resolver assigns each local variable a position within its scope and records its scope depth. The interpreter then uses this information to access the variable directly. Global variables continue to use name-based lookup.

This approach reduces the work needed to access local variables while preserving support for nested functions and closures.